Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026

Week 4 Exercise

Objective 1: Conceptual Foundation

Apache Spark is a widely used big data framework. At its core, spark enhances data processing. The driver (coordinator) is Spark Context. It plugs into different cluster managers such as yarn or Kubernetes. The manager decides how CPU and ram is assigned. Finally, the worker nodes run multiple tasks. The cumulative architecture is that driver coordinates, the manager distributes resources, and the executors run the tasks.

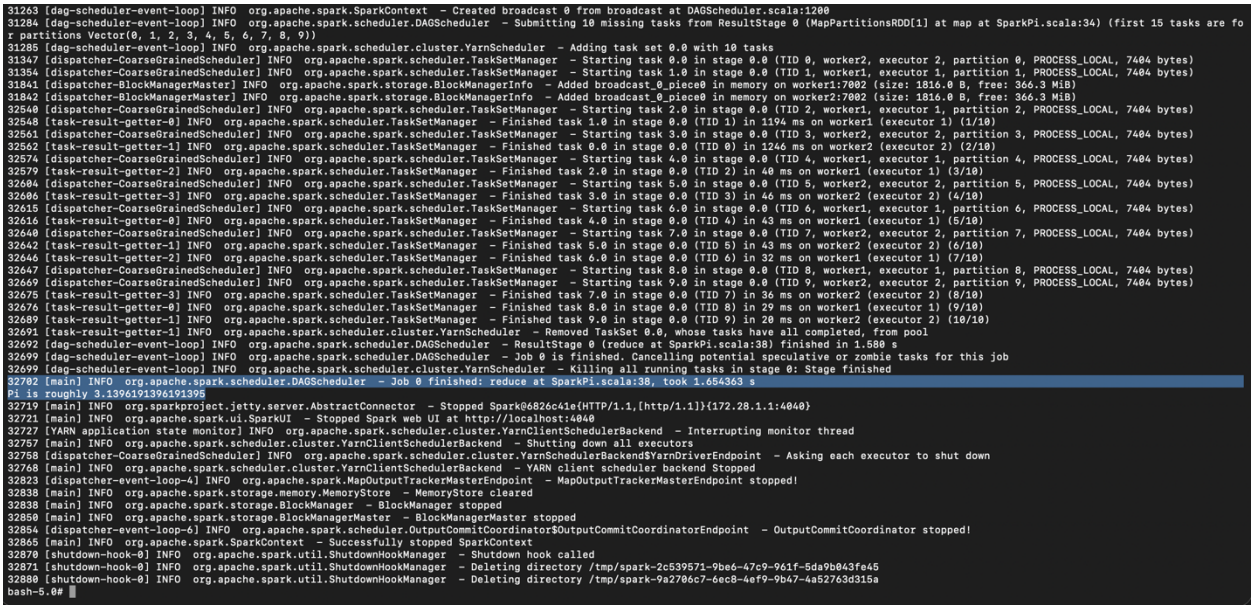
RDD is a fundamental concept to spark. RDD stands for resilient distributed dataset. Simply put, RDD allows spark to operate with redundancy to make the data fault tolerant. DAG executes your code across the structure. Spark itself WILL NOT execute until you call the execute command. This is ideal because it allows Spark to be "lazy" and formula a plan for the command across the cluster. A DAG is constructed which executes the code across the cluster. There is also in memory processing. This means that data is retained within memory, allowing spark to massively outperform MapReduce.

Past exercises used Hadoop map reduce. Compared to Spark, Map Reduce required reading/writing to disks whereas Spark did most of the work in memory. Spark's memory utilization saves resources and speeds up performance. If it feels like Spark is doing things smarter, it's because Hadoop/MapReduce are 1st generation computing whereas Spark is 2nd generation.

Objective 2 – Running SparkPi with PySpark

This is Apache Spark execution output showing the DAG scheduler creating stages, assigning tasks to executors, and tracking each task as it runs and finishes. After all stages complete successfully, Spark shuts down the context, stops executors, and cleans up temporary resources, indicating the job ended normally without errors.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026



Objective 3 – Spark Scala Shell

This shows the Spark interactive shell starting up on YARN, initializing a SparkContext and SparkSession, and connecting to the Spark web UI. You then create a local Scala collection of random numbers, parallelize it into an RDD, and trigger execution with a take(100).foreach(println), which causes Spark to run a job and print the first elements.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026

```
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
0    [main] WARN  org.apache.hadoop.util.NativeCodeLoader  - Unable to load native-hadoop library for your platform
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://localhost:4040
Spark context available as 'sc' (master = yarn, app id = application_1767981469663_0025).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.0.0
      /_/

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0_275)
Type in expressions to have them evaluated.
Type :help for more information.

scala>

scala>

scala> val numNumbers = 10000
numNumbers: Int = 10000

scala> val numbers = (1 to numNumbers).map(_ => scala.util.Random.nextInt(1000))
numbers: scala.collection.immutable.IndexedSeq[Int] = Vector(457, 658, 842, 434, 634, 626, 642, 381, 977, 141, 454,
 13, 555, 311, 286, 485, 392, 81, 294, 130, 851, 702, 239, 64, 149, 878, 150, 638, 169, 109, 464, 165, 739, 704, 27
, 126, 980, 905, 75, 730, 384, 131, 453, 603, 43, 847, 240, 532, 886, 587, 627, 939, 37, 11, 609, 900, 386, 492, 20
 268, 521, 429, 71, 741, 539, 660, 706, 823, 114, 632, 724, 900, 748, 275, 79, 955, 246, 813, 656, 381, 238, 874, 2
51, 503, 476, 847, 171, 58, 696, 407, 417, 749, 479...

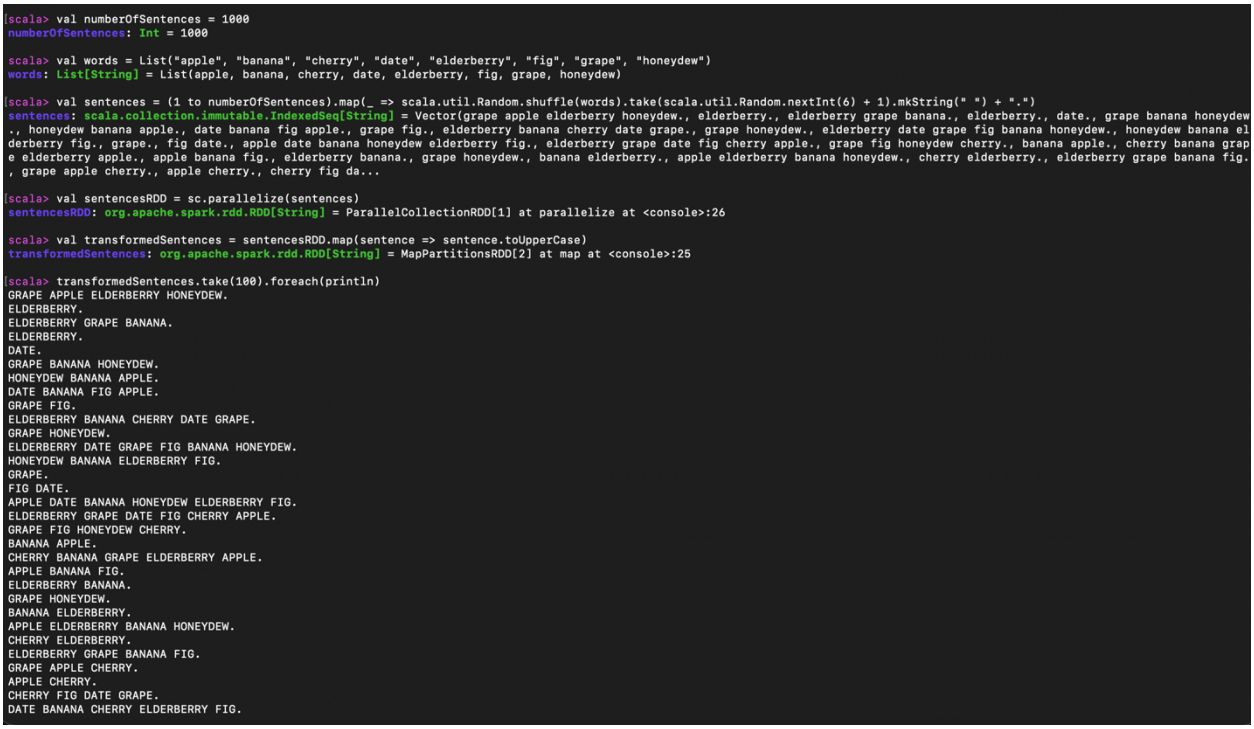
scala> val numbersRDD = sc.parallelize(numbers)
numbersRDD: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at <console>:26

scala> numbersRDD.take(100).foreach(println)
457
658
842
434
634
626
642
381
977
141
454
573
683
261
776
499
```

Objective 4 – Custom Transformation with Scala

This shows a Spark Scala REPL session where you generate random sentences from a word list, parallelize them into an RDD, and apply a map transformation to convert each sentence to uppercase. The final take(100).foreach(println) action triggers execution and prints the transformed results, demonstrating a basic RDD transformation and action workflow.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026



Objective 5 – Spark IDE + week4.py

This script starts a Spark session, generates 1,000 random sentences on the driver, and parallelizes them into an RDD so Spark can process them in parallel. It then applies a map transformation that converts each sentence into a tuple containing the sentence itself, the total word count, and the number of unique words, and finally triggers execution with

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026

take(100) to print sample results before shutting Spark down.

Week 4 Vance White Paper.pdf | vance_week4_init.py

winter2025 > dsc650_big_data > week4 > vance_week4_init.py > ...

```python
from pyspark.sql import SparkSession
import random

# Start Spark
spark = SparkSession.builder.appName("SentenceGenerator").getOrCreate()
sc = spark.sparkContext

# Word list
words = ["apple", "banana", "cherry", "date", "elderberry", "fig", "grape", "honeydew"]

# Generate sentences on the driver
num_sentences = 1000
sentences = [
    " ".join(random.sample(words, random.randint(1, 6))) + "."
    for _ in range(num_sentences)
]

# Parallelize into RDD
sentences_rdd = sc.parallelize(sentences)

# Transformation: add features to each sentence
# this turns each sentence into a tuple of (sentence, total_words, unique_words)
transformed = sentences_rdd.map(
    lambda s: (
        s,
        len(s.replace(".", "").split()),                # total words
        len(set(s.replace(".", "").split()))            # unique words
    )
)


# Show some results (will go to YARN driver logs in cluster mode)
for line in transformed.take(100):
    print(line)

spark.stop()

```

After wget'ing the file from github into the VM, we ran using spark-submit. This confirms that my Spark job actually ran inside the container without errors and that Spark was able to execute my code end to end. The output shows my sentences being processed in parallel and correctly transformed into tuples with total and unique word counts, which tells me the script was found, executed, and completed successfully.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026

```
28245 [dag-scheduler-event-loop] INFO  org.apache.spark.scheduler.cluster.YarnScheduler  - Killing all running tasks in stage
28245 [Thread-5] INFO  org.apache.spark.scheduler.DAGScheduler  - Job 0 finished: runJob at PythonRDD.scala:154, took 1.32587
('cherry banana fig.', 3, 3)
('elderberry date apple grape honeydew cherry.', 6, 6)
('honeydew.', 1, 1)
('fig elderberry.', 2, 2)
('apple elderberry fig grape cherry.', 5, 5)
('elderberry.', 1, 1)
('banana cherry apple grape.', 4, 4)
('honeydew apple elderberry fig banana cherry.', 6, 6)
('grape date banana apple cherry.', 5, 5)
('elderberry fig grape honeydew date.', 5, 5)
('honeydew date apple.', 3, 3)
('honeydew.', 1, 1)
('apple cherry banana date elderberry grape.', 6, 6)
('date.', 1, 1)
('cherry fig apple date.', 4, 4)
('elderberry.', 1, 1)
('date fig honeydew apple.', 4, 4)
('fig date cherry grape honeydew.', 5, 5)
('banana apple elderberry grape cherry date.', 6, 6)
('cherry apple date fig elderberry.', 5, 5)
('grape elderberry.', 2, 2)
('honeydew apple cherry grape elderberry.', 5, 5)
('fig.', 1, 1)
('honeydew apple date grape.', 4, 4)
('apple date fig grape.', 4, 4)
('date cherry.', 2, 2)
('honeydew date fig banana apple.', 5, 5)
('fig cherry honeydew elderberry apple banana.', 6, 6)
('banana apple.', 2, 2)
('elderberry.', 1, 1)
('honeydew grape banana apple.', 4, 4)
('banana date.', 2, 2)
('apple banana elderberry grape fig.', 5, 5)
('elderberry date grape apple banana fig.', 6, 6)
('elderberry date cherry fig grape honeydew.', 6, 6)
('fig grape elderberry.', 3, 3)
('fig date cherry grape banana.', 5, 5)
('apple grape date fig elderberry.', 5, 5)
('date banana apple honeydew.', 4, 4)
('grape cherry.', 2, 2)
('cherry apple grape elderberry banana.', 5, 5)
('banana grape honeydew.', 3, 3)
('apple date fig.', 3, 3)
('banana elderberry.', 2, 2)
('fig cherry honeydew banana apple date.', 6, 6)
('apple grape.', 2, 2)
('elderberry.', 1, 1)
('banana apple date fig.', 4, 4)
('banana grape.', 2, 2)
('cherry grape honeydew fig.', 4, 4)
('banana apple cherry date honeydew elderberry.', 6, 6)
('apple.', 1, 1)
('apple.', 1, 1)
('elderberry banana date honeydew cherry.', 5, 5)
```

Next we can ls the tmp dir to confirm the output was saved.

```
bash-5.0# hdfs dfs -ls /tmp/week4_output
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-01-19 05:53:56,206 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 3 items
-rw-r--r--   1 root supergroup          0 2026-01-19 05:49 /tmp/week4_output/_SUCCESS
-rw-r--r--   1 root supergroup      17354 2026-01-19 05:49 /tmp/week4_output/part-00000
-rw-r--r--   1 root supergroup      18054 2026-01-19 05:49 /tmp/week4_output/part-00001
bash-5.0#
```

And finally we can cat the file to confirm its contents. This confirms that my Spark job successfully wrote its output to HDFS and that I can read it back from the cluster

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 10, 2026

filesystem. Seeing the tuples printed by `hdfs dfs -cat` tells me the data was actually persisted in HDFS, not just computed in memory, so the full pipeline—Spark execution, HDFS write, and HDFS read—worked correctly inside the container.

